Wireless Emergency Traffic Signal priority System

D. Vani Venkata Priya
Dept. of Electronics and Communication Engineering
RGUKT

***Abstract*—In this project, a wireless emergency-priority traffic signal system is developed using an STM32 microcontroller and HC-12 RF communication modules. The system ensures that emergency vehicles such as ambulances, fire trucks, and police vehicles can pass through intersections without delay by overriding normal traffic light sequences. An Arduino module simulates the emergency vehicle side, where the driver presses a push button to indicate the approaching lane. This lane information is transmitted wirelessly using the HC-12 module at 433 MHz and received by the STM32, which instantly switches the corresponding traffic light to green while keeping all other lanes red. Once the emergency clears, the system returns to normal traffic light operation. This low-cost, offline solution eliminates the need for internet or camera-based systems, making it a scalable and reliable option for smart traffic management.**

***Index Terms*—STM32, Arduino, HC-12, Traffic Lights, Emergency Vehicle Priority, Wireless Communication**

I. Introduction

One of the biggest challenges in growing cities is heavy traffic congestion, which becomes a life-threatening issue when it blocks emergency vehicles like ambulances and fire trucks. In critical medical situations, such as cardiac arrests or severe accidents, the ”Golden Hour”—the first hour after injury—is vital. However, traditional traffic lights operate on fixed time loops. They are ”blind” to the situation on the road and cannot pause or change their sequence for an emergency. This forces ambulance drivers to either wait helplessly in gridlock or drive dangerously through red lights, risking further accidents.

While modern ”Intelligent Traffic Systems” do exist, they typically rely on expensive technologies like high-definition cameras, image processing, or continuous internet connections (IoT). These systems are not only costly to install but also prone to failure. If the internet network goes down or becomes slow—which often happens during storms or in developing areas—the entire smart system stops working.

To address these limitations, this project introduces a robust, offline solution using the STM32 microcontroller and HC-12 wireless modules. The goal is to provide a ”Green Wave” for emergency vehicles without relying on complex cloud infrastructure. The system functions as a direct, long-range remote control: as an emergency vehicle approaches, it wirelessly signals the traffic controller to immediately switch that specific lane to green.

We specifically chose the HC-12 RF module for this design because of its reliability and long range (up to 1 km). Unlike Wi-Fi or GSM-based systems, this setup works completely independently of cellular networks. This makes the proposed system a practical, low-cost, and highly reliable alternative for cities and rural regions where internet stability cannot be guaranteed.

II. Literature Survey

The population of India is growing, the number of vehicles is also increasing tremendously. This makes it difficult to manage time and priority of emergency vehicles like ambulance, fire brigade, police vehicles etc.In [1] the proposed model basically works on properly managing the traffic and clear the congestion of vehicles in order to make clear way for emergency vehicles and reduce their effort and time to reach the destination. More importantly, traffic congestion can also be life-threatening when emergency vehicles (EVs) must travel through congested areas, which may prevent them from reaching accident spots on time. In every major city or state, services offered by ambulance, fire, and other emergency services usually have a strict target response time and quality of service (QoS) metric to meet while attending incidents. Congestion can severely hinder the response time and QoS in rendering those services [2]. Over the years, traffic lights have been used to manage traffic at road intersections. Though relatively effective, the vehicular queues that build up at each intersection being managed by a traffic light and the subsequent delay thereof, could have adverse effects on medical emergency vehicles (EVs). Reports have shown that queues at traffic intersection can increase travel times of medical EVs by an average of 20%. This in many instances could mean the difference between life and death. Prioritizing EVs could be a potential solution to this challenge [3]. Due to traffic congestion High Priority Vehicles (HPV) also get stuck in traffic which results delay in their services. HPV like ambulances, fire brigade etc. have to serve various causalities. It is very important for HPV to reach on time. There is a need of system that aims to provide path to HPV to reach as quickly as possible [4]. Manual control of traffic hasnot proved to be efficient,also a predefined signal timing for the signal at all circumstances whether high or low traffichas not done any ease to the problem. Existing systems that use RF modules can only be usedfor one vehicle system at a time [5]. The proposed work provides priority based approach.The system will also control the traffic light in the absence of emergency vehicles. All the traffic lights are controlled by the central unit.

III. System Architecture

The system is divided into two main sections: the transmitter unit inside the vehicle and the receiver unit at the traffic pole.

A. Emergency Vehicle Unit

Hardware Setup: This unit is built using an Arduino, a set of push buttons, and an HC-12 transmitter module.
Driver Input: As the emergency vehicle approaches the intersection, the driver simply presses the button corresponding to their current lane (e.g., Lane 1, Lane 2, etc.).
Data Transmission: The Arduino reads this button press, creates a specific command (like ”L2”), and sends it wirelessly via the HC-12 module to the traffic controller.

B. Traffic Signal Controller Unit

Hardware Setup: This unit controls the intersection lights using an STM32 microcontroller connected to an HC-12 receiver and the traffic LEDs.
Signal Reception: The HC-12 receiver picks up the wireless command from the vehicle and passes the data to the STM32.
Priority Switching: Upon receiving the signal, the STM32 immediately pauses the normal traffic cycle and turns the requested lane green.
Safety Lock: While the emergency lane is green, the system forces all other lanes to stay red to ensure the intersection is clear.
Restoring Normal Flow: Once the emergency signal stops or a preset time passes, the controller automatically switches back to its regular traffic sequence.

This simple architecture ensures that communication happens in real-time, allowing the system to make quick decisions for the safety of the emergency response team.

IV. Working Principle

A. Normal Mode

Under normal conditions, the system operates just like a standard traffic signal. The STM32 microcontroller runs a continuous loop where it activates the lanes in a fixed sequence: Lane 1 $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red $\rightarrow$ Lane 2 $\rightarrow \ldots \rightarrow$ Lane 4, and repeats.

B. Emergency Mode

Trigger: The driver presses the specific push button on their dashboard unit (the Arduino transmitter).
Transmission: The Arduino sends a coded command wirelessly via the HC-12 module (for example, sending ”L3” for Lane 3).
Action: The STM32 at the traffic pole receives this command and immediately interrupts the normal cycle. It turns the requested lane Green instantly while forcing all other lanes to stay Red.
Reset: Once the emergency input stops (or after a set delay), the system automatically goes back to the Normal Mode sequence.

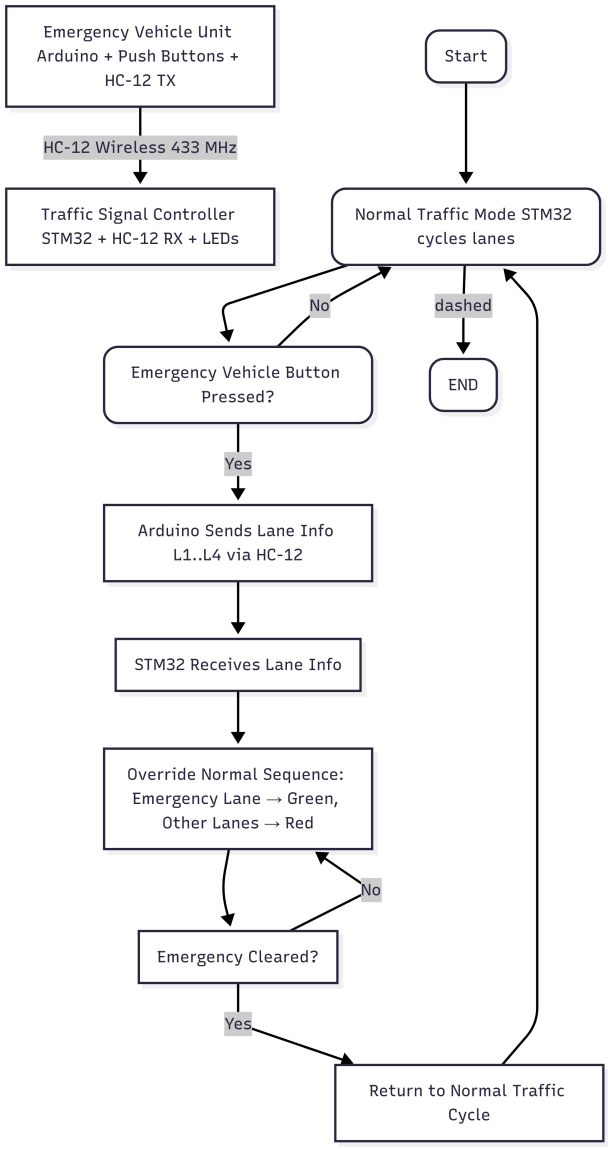

Fig. 1. Flowchart

C. Traffic Light Setup

For the visual output, we used a total of 12 LEDs (Red, Yellow, and Green for each of the four lanes). These are directly controlled by the GPIO pins of the STM32. This dual-mode design ensures that the system can handle daily traffic smoothly while being ready to switch to priority mode at any second.

D. Packet Construction and Validation

The packet is constructed on the Arduino (Transmitter) as a character array and includes redundant delimiters and a checksum, which is essential for V2I systems. The total length of 8 bytes includes the start/stop signals.Header and Delimiters: The packet begins with the **START_DELIMITER** (\$) and ends with a carriage return \r followed by a line feed \n. These markers define the exact boundaries for the STM32's UART

interrupt handler, ensuring precise data assembly.Checksum Validation (LRC): The two bytes at indices 4 and 5 represent the 8-bit **Checksum** (C1C2), which is generated by summing the ASCII values of the three critical payload fields (Type + Priority + Lane). The STM32 must perform this same calculation and verify the result against the received C1C2 characters. This validation step is crucial for guaranteeing signal integrity and filtering out corrupted packets.Payload: The core control information—the Vehicle Type (for base identification), the Priority Level (for dynamic scheduling), and the Lane ID (for output control)—is extracted only after the checksum is validated, ensuring safety.

TABLE I
PACKET FORMAT

Index	Field	Size	Example	Purpose
0	Start Delimiter	1	$	Marks the beginning of a valid packet
1	Vehicle Type	1	A	Type of emergency vehicle (Ambulance)
2	Priority Level	1	3	Used by Dynamic Priority Algorithm
3	Lane ID	1	1–4	Specifies which lane receives priority
4	Checksum Char 1	1	C1	High nibble of 8-bit checksum
5	Checksum Char 2	1	C2	Low nibble of 8-bit checksum
6	Carriage Return	1	\r	Line termination control character
7	Stop Delimiter	1	\n	Marks end of packet and triggers processing

V. HARDWARE IMPLEMENTATION

To demonstrate the working of the project, we built two separate hardware modules: a portable transmitter unit (representing the ambulance) and a stationary receiver unit (representing the traffic signal).

A. Transmitter Unit (Emergency Vehicle Side)

This unit is designed to be handheld or dashboard-mounted. We built it using an Arduino board because of its simplicity in handling button inputs.
Microcontroller: We used an Arduino Uno as the central processor.
Input Buttons: Four push buttons are connected to the digital input pins. Each button corresponds to a specific lane (Lane 1 through Lane 4). We used internal pull-up resistors in the code to keep the wiring simple and avoid floating signals.
Wireless Module: The HC-12 module is connected to the Arduino's software serial pins.

Operation: When we press a button, the Arduino detects the input and immediately sends a short character string (like "L1") to the HC-12, which broadcasts it into the air.

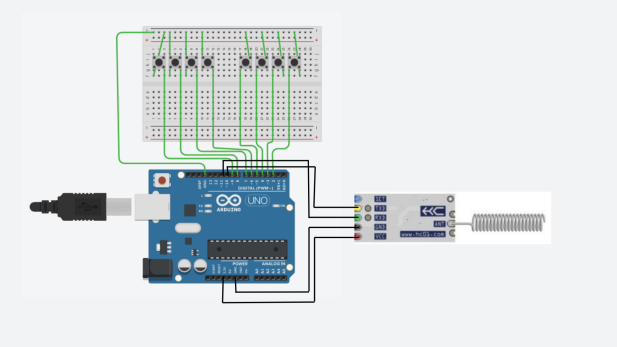

Fig. 2. Transmitter unit

B. Receiver Unit (Traffic Controller Side)

This is the main control unit located at the intersection. We used the STM32F103RB (Nucleo board)here because it is faster and has more I/O pins to handle multiple traffic lights compared to a standard Arduino.
Microcontroller:The STM32 serves as the master controller.

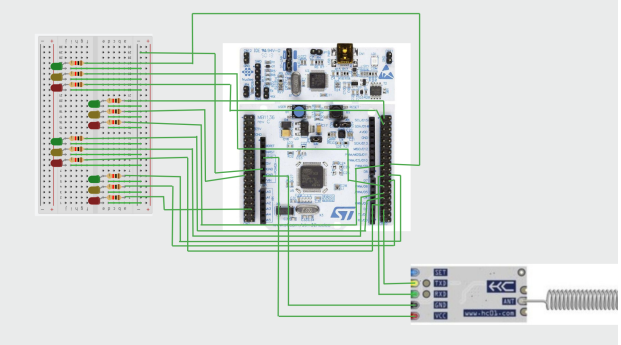

Fig. 3. Receiver Unit

Wireless Reception: A second HC-12 module is wired to the hardware UART (TX/RX) pins of the STM32 (typically PA9 and PA10). It constantly listens for incoming data.

Traffic Lights: We simulated the traffic signals using 12 LEDs (Red, Yellow, Green for 4 lanes). These are connected to the GPIO output pins of the STM32 via current-limiting resistors to prevent burning out the LEDs.

Power Supply: Since the HC-12 can draw significant current during transmission, we ensured both units are powered by a stable 5V source (using USB or a 9V battery with a regulator) to prevent the microcontrollers from resetting.

C. Connection Setup

For the communication to work, we configured both HC-12 modules to the same baud rate (9600 bps) and the same channel. This ensures they can "hear" each other clearly without interference from other devices

VI. Protocols Used

A. UART Communication

We used the UART protocol to establish a link between the Arduino and the STM32. The HC-12 modules essentially act as a ”wireless wire” or bridge. When the Arduino sends simple text data (like “L1” or “L2”), it travels serially through the HC-12 and is read by the STM32, which then decides which lane to open.

B. HC-12 RF Transmission (433 MHz)

This module handles the wireless part of the project using the 433 MHz frequency. We chose this because it offers long-range communication and works completely offline. Since it uses simple serial commands, we didn’t need to write complex networking code (like Wi-Fi or Bluetooth stacks), making the system faster and easier to build.

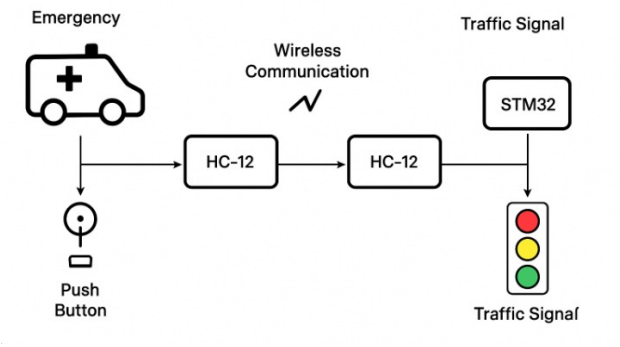

Fig. 4. Communication between transmitter unit and receiver unit

C. GPIO Control (STM32)

The physical traffic lights (LEDs) are controlled directly by the General Purpose Input/Output (GPIO) pins of the STM32. By toggling these pins in the code, the microcontroller can switch the lights on or off instantly. This direct connection ensures there is no delay when switching from Red to Green during an emergency.

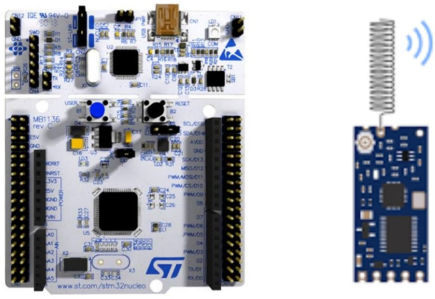

Fig. 5. Hardware Used

VII. System Integration and Challenges

A. Integration of Components

Bringing the system together involved linking the vehicle’s transmitter unit with the traffic controller. The data flow starts at the Arduino, travels wirelessly through the HC-12, is processed by the STM32, and finally triggers the LEDs. The biggest hurdle during integration was writing the code logic so that the STM32 could smoothly switch between the boring, repetitive traffic cycle and the sudden, high-priority emergency mode without crashing or getting stuck.

B. Challenges and Solutions

During the build and testing phase, we faced several issues that required specific fixes:

1) **Interference on the Wireless Line:**
   Issue: Since the HC-12 uses the common 433 MHz frequency, we sometimes picked up ”noise” or interference that confused the receiver.
   Solution: We improved the code by wrapping our data messages with specific start and stop markers. This way, the receiver ignores any signal that doesn’t look exactly like our specific command.
2) **Returning to Normal Traffic:**
   Issue: After the emergency vehicle passed, the system didn’t always know how to restart the normal cycle smoothly, sometimes leading to conflicting green lights.
   *Solution:*We implemented a simple ”timeout” timer. Once the emergency signal stops, the system waits for a few seconds and then resets the loop from the beginning, ensuring a safe transition.
3) **Two Emergencies at Once:**
   Issue: We realized a potential problem: what if an ambulance and a fire truck approach from different lanes at the same time?
   Solution: We added a priority logic in the code (for example, prioritizing Ambulances first) so the system can decide which lane needs to open most urgently.

VIII. Comparative Analysis

A. Camera-Based Systems

While effective in some places, camera-based systems are very expensive because they require high-processing computers and complex image recognition software. A major practical flaw is their sensitivity to the environment; they often struggle to detect vehicles accurately during heavy rain, fog, or low-light conditions.

B. IoT (Wi-Fi/GSM) Systems

Systems that run on the cloud or 4G/5G networks are popular, but they introduce a ”point of failure”: the internet connection. If the network is congested or goes down, the signal to the traffic light will be delayed or lost. In emergency situations, even a few seconds of ”lag” due to a slow connection can be dangerous.

C. Proposed HC-12 System (Our Solution)

Our system is designed to solve the specific drawbacks of the options above.

Offline Reliability: Unlike IoT, it creates a direct radio link that works without any internet or monthly data plans. Cost and Simplicity: It uses standard, affordable components (STM32 and HC-12) instead of expensive cameras.

Practicality: Because it doesn't rely on external infrastructure, it is much easier to deploy in developing regions or rural areas where high-speed internet isn't always available.

TABLE II
COMPARISON WITH EXISTING SYSTEMS

Feature	**Camera / AI [1]**	**IoT / Cloud [2]**	**Basic RF [5]**	**Proposed (STM32)**
Tech	HD Cameras	Web Server	Simple RF	**STM32 + HC-12**
Dependency	Visibility	Internet	Line of Sight	**None (Offline)**
Cost	Very High	High (Data)	Low	**Low**
Weather	Fails in Fog/Rain	Net Lag	Good	**High Reliability**
Multi-Vehicle	Yes (AI)	Yes (Server)	No	**Yes (Priority Queue)**
Priority	AI Logic	Server Logic	FCFS	**Amb > Fire**

IX. FUTURE ENHANCEMENTS

While our current prototype works well for single intersections, there is a lot of room to make it smarter and more robust in the future:

1) **The "Green Corridor" Concept:** By linking multiple traffic lights together, we could create a synchronized "Green Corridor." This would allow an ambulance to pass through 5 or 6 intersections without stopping once, drastically reducing travel time to the hospital.
2) **Security Upgrades:** Since RF signals can technically be mimicked, a commercial version of this project would need encryption (like rolling codes) to prevent unauthorized people from triggering the green light.
3) **Pedestrian Warning Systems:** We can add a loud buzzer or a digital text display at the intersection. When the emergency mode is activated, it would warn pedestrians and other drivers with a "STOP - AMBULANCE APPROACHING" alert to prevent accidents during the signal change.
4) **Green Energy:** To make the system self-sustaining, especially in rural areas, we could power the simplified traffic controller using solar panels.

X. CONCLUSION

This project successfully demonstrates that "smart" traffic systems don't always need expensive cameras or unstable internet connections to work effectively. By using the STM32 microcontroller and HC-12 wireless modules, we built a reliable system that gives immediate priority to emergency vehicles.

The key takeaway from this work is that a hardware-based, offline solution is often more practical for developing regions than complex cloud-based systems. It is cost-effective, easy to maintain, and most importantly, it saves the one thing that matters most in an emergency: time. This prototype serves as a solid foundation for future developments in low-cost urban traffic management.

The prototype validates the effectiveness of RF-based traffic control and serves as a foundation for scaling into more advanced smart traffic management systems in the future.

REFERENCES

[1] S. V. Bhate, P. V. Kulkarni, S. D. Lagad, M. D. Shinde, and S. Patil, "Iot based intelligent traffic signal system for emergency vehicles," in *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*, 2018, pp. 788–793.

[2] G. Karmakar, A. Chowdhury, J. Kamruzzaman, and I. Gondal, "A smart priority-based traffic control system for emergency vehicles," *IEEE Sensors Journal*, vol. 21, no. 14, pp. 15 849–15 858, 2021.

[3] O. Ajayi, A. Bagula, I. Chukwubueze, and H. Maluleke, "Priority based traffic pre-emption system for medical emergency vehicles in smart cities," in *2020 IEEE Symposium on Computers and Communications (ISCC)*, 2020, pp. 1–7.

[4] D. Dang, J. Tanwar, and S. Masood, "A smart traffic solution for high priority vehicles," in *2015 1st International Conference on Next Generation Computing Technologies (NGCT)*, 2015, pp. 466–470.

[5] B. A. Anandh, A. S. Ganesh, and P. Nandakumar, "Smart traffic light control system for emergency vehicles using rf module," in *Futuristic Trends in Electronics & Instrumentation Engineering*, ser. IIP Series, vol. 3, no. 1. IIP Publications, May 2024, pp. 208–216. [Online]. Available: https://www.doi.org/10.58532/V3BDEI1P5CH3